

Assignment 1 (100 Marks)

COMP2123 Data Structures and Algorithms
Semester 2, 2026

This assignment is due in **Week 5** and must be submitted through Gradescope.

Work in **groups of three**. One group member must upload the PDF and add the other two group members to the Gradescope submission.

Submit a PDF containing typed answers. Handwritten answers and PDFs consisting of scanned images of text will not be accepted. The PDF must contain selectable text so that it can be read by Turnitin.

Type the student IDs of all three group members at the top left of the first page. Do not type any names. Do not include a cover page. Submit only the answers and do not copy the problem statements into the submission.

The maximum submission length is **five pages**. If a submission contains more than five pages, only the first five pages will be reviewed.

For every problem, begin with a clear description of the main idea. Give enough detail for the algorithm to be implemented, prove that the algorithm is correct, and analyse its worst case running time and additional space usage. Pseudocode is optional. Pseudocode without an explanation in plain English is not sufficient.

The submitted work must be produced by the registered group. Any source outside the lecture, tutorial, and recommended textbook material must be cited.

Problem 1.

[20 points]

Nested cryptographic transcript

A cryptographic protocol may execute several proof components inside one another. An audit system records the beginning and end of each component in a transcript

$$E[0], E[1], \dots, E[n-1].$$

Each entry has one of the following forms:

$$\text{open}(k),$$

meaning that the proof component with identifier k begins, or

$$\text{close}(k),$$

meaning that the proof component with identifier k ends.

Every identifier appears in at most one open event and at most one close event.

A transcript is *valid* when both of the following conditions hold:

1. Every $\text{close}(k)$ event matches the most recently opened component that has not already been closed.
2. Every opened component is closed before the transcript ends.

The number of components that are open at the same time is called the current nesting depth. The largest nesting depth reached anywhere in the transcript is called the *maximum nesting depth*.

The algorithm must return one of the following outputs:

1.

`(valid, d)`,

where d is the maximum nesting depth, if the transcript is valid.

2.

`(invalid, i)`,

where i is the index of the first close event that cannot be matched correctly. Transcript indices start at 0.

3.

`incomplete`,

if every close event is matched correctly, but one or more components remain open when the transcript ends.

The transcript may be empty. The empty transcript is valid and has maximum nesting depth 0.

Examples. For the transcript

$$E = [\text{open}(7), \text{open}(3), \text{close}(3), \\ \text{open}(9), \text{close}(9), \text{close}(7)],$$

the correct output is

`(valid, 2)`.

For the transcript

$$E = [\text{open}(7), \text{open}(3), \text{close}(7), \text{close}(3)],$$

the correct output is

`(invalid, 2)`,

because component 3 must be closed before component 7.

For the transcript

$$E = [\text{open}(7), \text{open}(3), \text{close}(3)],$$

the correct output is

`incomplete`,

because component 7 remains open.

Design an algorithm that produces the required output in $O(n)$ worst case time.

- Describe the algorithm in plain English.
- Prove that the algorithm returns the correct output.
- Analyse the worst case running time and additional space used by the algorithm.

Problem 2.

[30 points]

Product recall in a catalogue tree

A retailer organises its products using a rooted catalogue tree T containing n nodes. Every node has a distinct label.

Each leaf represents one product. Every leaf is marked either *safe* or *recalled*. Internal nodes represent product categories and do not have a mark. At least two leaves are recalled.

Issuing a recall at a node v removes every product represented by a leaf in the subtree rooted at v .

A node v is a *valid recall node* if the subtree rooted at v contains every recalled product in the entire catalogue.

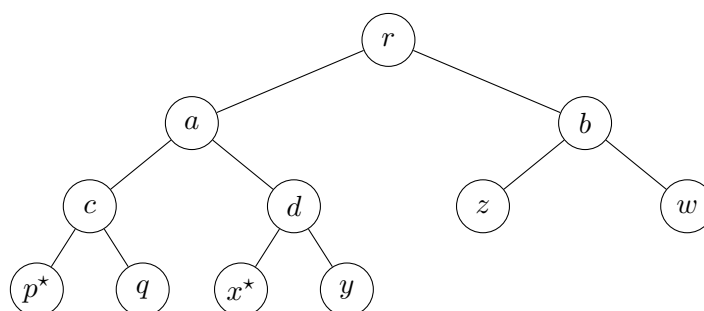
The depth of a node is the number of edges on the path from the root to that node. Among all valid recall nodes, the retailer chooses the node of maximum depth. This keeps the recall within the smallest possible part of the catalogue.

The algorithm must return an ordered pair (v, s) , where

- v is the valid recall node of maximum depth
- s is the number of safe product leaves in the subtree rooted at v

The value s may be 0.

Example. In the following tree, a star marks a recalled product.



The recalled products are p and x . The subtree rooted at a contains both recalled products. Neither the subtree rooted at c nor the subtree rooted at d contains both recalled products. Therefore, a is the valid recall node of maximum depth.

The safe products in the subtree rooted at a are q and y . The correct output is therefore

$(a, 2)$.

The tree is not assumed to be binary. The children of a node may be examined using the Tree ADT operations from the lecture. The label of any node and the mark of any leaf can be inspected in $O(1)$ time.

Design an algorithm that produces the required output in $O(n)$ worst case time.

- (a) Describe the algorithm in plain English.
- (b) Prove correctness, that the returned node is the valid recall node of maximum depth. Also prove that the returned number of safe products is correct.
- (c) Analyse the worst case running time and additional space used by the algorithm.

Problem 3.

[50 points]

Risk threshold audit in a certificate registry

A certificate registry stores n active certificates in a binary search tree T . Each certificate has

- 1. a distinct integer serial number s
- 2. a positive integer risk score $r(s)$

The binary search tree is ordered by serial number. Let h be the height of T when the tree is nonempty. The tree is not assumed to be balanced.

A security auditor chooses an integer lower bound a and a positive integer risk threshold B . The audit begins with the first stored certificate whose serial number is at least a , and then examines certificates in increasing serial number order.

Define

$$\text{auditWindow}(a, B)$$

to return the ordered pair (x, m) , where x is the smallest serial number stored in T such that $x \geq a$ and

$$\sum_{\substack{s \text{ is stored in } T \\ a \leq s \leq x}} r(s) \geq B,$$

and where

$$m = \max_{\substack{s \text{ is stored in } T \\ a \leq s \leq x}} r(s).$$

Thus, x is the first serial number at which the accumulated risk from serial number a reaches the threshold, and m is the largest individual risk score encountered in that audit window.

If no such serial number x exists, the operation returns **None**. In particular, an empty tree returns **None**.

Example. Suppose the registry contains the following pairs, written as

$$(\text{serial number, risk score}) : (2, 4), \quad (5, 1), \quad (8, 6), \quad (11, 3), \quad (14, 5).$$

Starting from serial number 4, the examined risk scores are 1, 6, 3, 5. The threshold 7 is first reached at serial number 8, and the largest risk score encountered is 6. Therefore,

$$\text{auditWindow}(4, 7) = (8, 6).$$

Starting from serial number 6, the threshold 7 is first reached after examining serial numbers 8 and 11. Therefore,

$$\text{auditWindow}(6, 7) = (11, 6).$$

Some other correct outputs are

$$\text{auditWindow}(1, 5) = (5, 4),$$

$$\text{auditWindow}(9, 6) = (14, 5),$$

and

$$\text{auditWindow}(12, 6) = \text{None}.$$

If certificate 8 is removed, then

$$\text{auditWindow}(4, 7) = (14, 5).$$

If a new certificate with serial number 7 and risk score 8 is then inserted, the correct output becomes

$$\text{auditWindow}(4, 7) = (7, 8).$$

Assume that the standard binary search tree implementations of $\text{put}(s, r)$ and $\text{remove}(s)$ from the lecture are already available. Every call to $\text{put}(s, r)$ inserts a serial number that is not currently stored in the tree. Every call to $\text{remove}(s)$ removes a serial number that is currently stored in the tree. When the standard removal algorithm replaces a node entry by its inorder successor entry, both the serial number and the risk score are copied.

A constant number of additional integer fields may be stored at each node that contains a certificate. The standard pointer operations for insertion and removal do not need to be reproduced. It is sufficient to explain how the additional information is maintained in every case of the standard insertion and removal algorithms.

All risk scores, sums of risk scores, and other stored integer values fit in fixed size integers. Each arithmetic operation and comparison takes $O(1)$ time.

Requirements.

1. If the tree is nonempty, $\text{auditWindow}(a, B)$ must run in $O(h)$ worst case time. If the tree is empty, it must run in $O(1)$ time.
2. put and remove must continue to run in $O(h)$ worst case time.

3. The total space used by the binary search tree must be $O(n)$.

- (a) Give an algorithm for `auditWindow( $a, B$ )`.
- (b) Prove that `auditWindow( $a, B$ )` returns the required pair (x, m) , or returns **None** exactly when no valid serial number x exists.
- (c) Analyse the worst case running time of `auditWindow( $a, B$ )`.